Branch and Bound

Search with a proof, not just a good answer

> A feasible solution says, "I can do this well." A bound says, "Nothing in this entire region can do better than that." Branch and bound combines these statements to discard whole families of decisions safely.

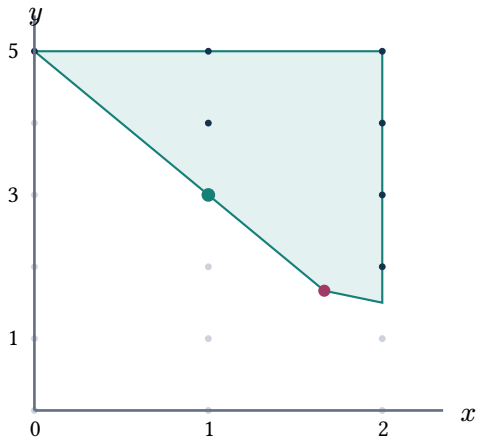
1. Why not just solve continuously and round?

Suppose two types of indivisible resource provide two services. One unit of type x supplies $(2, 1)$ units of service and costs 3; one unit of type y supplies $(1, 2)$ and costs 2. Both services need at least 5 units. At most two x units and five y units are available:

$$z^\star = \min(3x + 2y) \quad \text{subject to} \quad 2x + y \geq 5, \quad x + 2y \geq 5, \tag{1}$$

$$0 \leq x \leq 2, \quad 0 \leq y \leq 5, \quad x, y \in \mathbb{Z}. \tag{2}$$

If we allow fractional resources, the two service boundaries meet at $(x, y) = \left(\frac{5}{3}, \frac{5}{3}\right)$ with cost $\frac{25}{3} \approx 8.333$. This is the continuous **linear programming (LP) relaxation**. Its optimum is not an implementable decision, but its cost will be useful.



Shading: LP-feasible region
Dark dots: feasible integer choices
Pale dots: infeasible integer choices

LP optimum: $\left(\frac{5}{3}, \frac{5}{3}\right)$
Cost: $\frac{25}{3}$ (a lower bound)

Integer optimum: $(1, 3)$
Cost: 9 (to be certified)

Figure 1: The continuous region contains all integer choices, plus artificial fractional ones. A relaxation enlarges the search space; it does not round it.

Nearest-integer rounding gives $(2, 2)$, cost 10: feasible but not optimal. Rounding down gives $(1, 1)$, which violates both service constraints. Rounding is a **candidate-generation heuristic**, not a proof of feasibility or optimality. We will instead certify $(1, 3)$ without testing every choice.

Prerequisites: linear inequalities and the idea of optimizing a linear function over a polygon. All bounds below use **minimization** unless stated otherwise.

2. Two inequalities power the whole method

For a mixed-integer linear program (MILP), only selected components must be integers. Write

$$z^\star = \min_{u \in F} c^T u, \quad F = \{u : Au \le b, u_i \in \mathbb{Z} \ \text{ for } \ i \in I\}. \tag{3}$$

The other components are real. A **node** N stores the original constraints plus the restrictions inherited along its path from the root. Its integer-feasible set is F_N and its relaxed set is R_N, with $F_N \subseteq R_N$.

2.1. A relaxation gives a lower bound

Minimizing over more choices cannot increase the optimal value:

$$\ell_N := \min_{u \in R_N} c^T u \le \min_{u \in F_N} c^T u =: z_N^\star. \tag{4}$$

An infeasible relaxation proves F_N empty. For Equation 2, the root value $\frac{25}{3}$ can be certified without trusting a plot. Multiply the first service inequality by $\frac{4}{3}$ and the second by $\frac{1}{3}$ and add:

$$3x + 2y \ge \left(\frac{4}{3}\right)5 + \left(\frac{1}{3}\right)5 = \frac{25}{3}. \tag{5}$$

The point $\left(\frac{5}{3}, \frac{5}{3}\right)$ attains this value, proving LP optimality. This weighted sum is a small **dual certificate**.

2.2. A feasible solution gives an upper bound

An **incumbent** $\hat{u} \in F$ is the best original-feasible solution found so far. Its objective $U = c^T \hat{u}$ satisfies $z^\star \le U$. Initially, use $U = +\infty$ if no feasible solution is known. Here $(0, 5)$ is feasible, so start with $U = 10$. It need not lie in the node currently being processed.

2.3. Branching partitions decisions, not guesses

If an optimal LP solution $|u|$ has a fractional integer component $|u|_j$, create two children with the inherited constraints and respectively

$$u_j \le \lfloor |u|_j \rfloor, \quad \text{or} \quad u_j \ge \lceil |u|_j \rceil. \tag{6}$$

Every integer value belongs to exactly one child; the fractional LP solution belongs to neither. The real interval between floor and ceiling contains no integers, so nothing implementable is lost. At the root, split into $x \le 1$ and $x \ge 2$.

2.4. Three reasons to close a node (fathom or prune)

Reason	Certificate	Why no search remains
Infeasibility	The relaxation is infeasible.	There is no original-feasible choice here.
Bound	$\ell_N \ge U$.	Every choice costs at least the incumbent.
Integrality	An optimal LP point satisfies all required integrality constraints.	It attains both bounds for this node. Update the incumbent if better.

The bound rule includes equality because we seek **one** optimum. If the goal is to enumerate all optimal solutions, discarding ties can lose other optima. The integrality rule requires an **optimal** relaxed point, not just any integer point encountered during the LP solve [1].

> "No feasible point found" is not "infeasible." "This branch looks expensive" is not a lower bound. Every pruning decision needs a valid certificate.

3. A complete five-node search

Keep the starting incumbent $(0, 5)$, cost 10. We deliberately explore the right-hand branch first so that all three closing rules appear. Bounds in the tree are **raw LP values**, without the integer-objective strengthening on page 5.

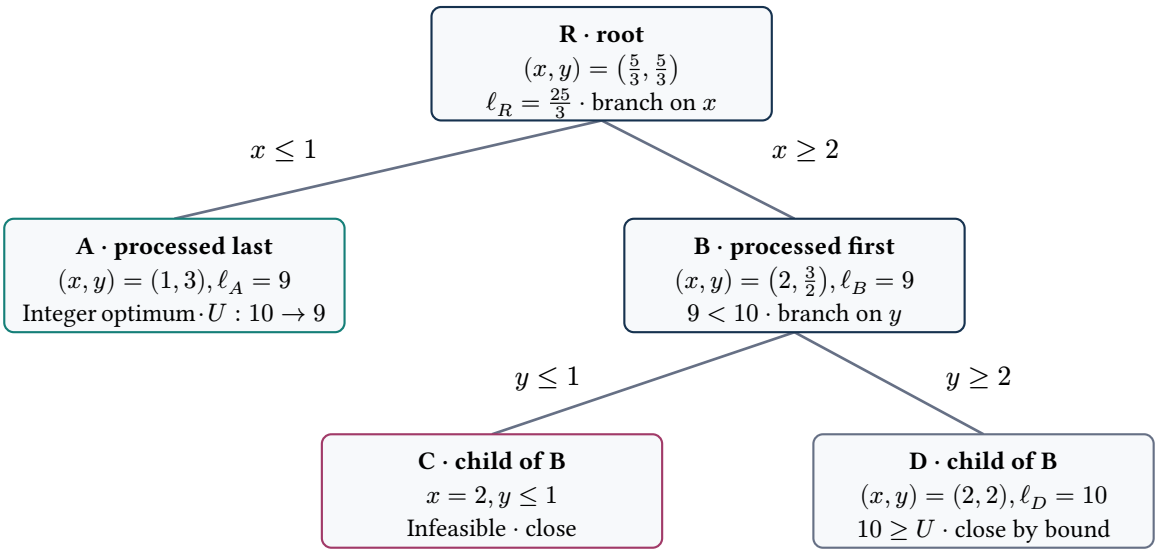


Figure 2: Processing order R, B, C, D, A. Every child inherits all constraints above it. Node letters identify regions, not the order of processing.

R. The root LP has cost $\frac{25}{3} < 10$, so it cannot be pruned. Its x value is fractional; branch using Equation 6.

B. Since the original upper bound is $x \le 2$, adding $x \ge 2$ fixes $x = 2$. Then $2x + y \ge 5$ requires $y \ge 1$, and $x + 2y \ge 5$ requires $y \ge \frac{3}{2}$. The minimum is at $y = \frac{3}{2}$, with cost 9. It is fractional and strictly below the incumbent, so split into $y \le 1$ and $y \ge 2$.

C. Here $x + 2y \le 2 + 2 = 4 < 5$. Even the relaxation is infeasible: close C.

D. Here $x = 2$, and the cheapest $y \ge 2$ is 2. Its LP bound is 10, equal to the incumbent. Close D by bound. The LP point also happens to be integer; the bound test is already sufficient.

A. With $x \le 1$, the first constraint gives $y \ge 5 - 2x$, hence

$$3x + 2y \ge 3x + 2(5 - 2x) = 10 - x \ge 9. \tag{7}$$

The integer point $(1, 3)$ attains 9 and meets the other constraint. Replace the incumbent by $(1, 3)$ and close A by integrality. No open nodes remain: **the incumbent is globally optimal, with cost 9**.

> If A is processed first, it immediately sets $U = 9$. Node B then closes by $\ell_B = 9 \ge U$ without creating C or D: three LP solves rather than five. Search order changes the work, not the validity of the final certificate.

4. The algorithm and why it cannot lose a better answer

The following exact-arithmetic version assumes finite bounds on all integer variables and that every feasible node LP has a finite, attained optimum. Continuous variables need not be integral. A node carries its complete path restrictions and a lower bound inherited from its parent until re-solved.

```
open = {root}; incumbent = none; U = +infinity
root.stored_lower_bound = -infinity
while open is not empty:
    remove a node N from open
    if its stored lower bound >= U: continue
    solve the LP relaxation of N
    if the LP is infeasible: continue
    ell = optimal LP objective; v = optimal LP point
    if ell >= U: continue
    if every integer-designated component of v is integer:
        incumbent = v; U = ell
        continue
    choose a fractional integer component v[j]
    add N with u[j] <= floor(v[j]) to open, bound = ell
    add N with u[j] >= ceil(v[j])  to open, bound = ell
if incumbent exists: return incumbent, globally optimal
else: return infeasible
```

Heuristics are optional. At any point, a heuristic may supply another original-feasible point and improve U. It must check every original constraint, not merely integrality. This can dramatically increase pruning without changing the proof. Conversely, a partial LP solve may supply a valid dual bound, but an arbitrary primal LP iterate is not a lower-bound certificate.

4.1. The coverage invariant

At every completed iteration, every original-feasible point is either in an open node, or in a closed region proved unable to beat the current incumbent. This holds initially because the root covers the whole problem.

- **Branching preserves coverage:** the two integer child sets partition the parent's integer set.
- **Infeasibility removes nothing feasible.**
- **Bound pruning removes no strict improvement:** for all $u \in F_N$, $c^T u \geq \ell_N \geq U$.
- **Integrality closes a solved region:** if an optimal relaxed point is original-feasible, its objective equals $z_N^\star$. Updating U retains a solution at least as good as every point there.

Later improvements only decrease U, so earlier pruning remains valid. When the open set is empty, nothing can beat the incumbent. With no incumbent, empty coverage instead certifies infeasibility.

4.2. Why termination needs assumptions

A branch on a fractional integer component strictly shrinks that component's integer interval in both children. Finite integer ranges imply finite depth and a finite tree. When every integer variable is fixed, the remaining LP solves the node exactly. Binary variables can still produce a full tree with up to $2^{n+1} - 1$ nodes for n variables: **correctness is not a polynomial-time guarantee**.

An unbounded LP relaxation is not, by itself, an unboundedness certificate for the integer problem: the integer problem may be infeasible. For example, $x = \frac{1}{2}$, $x \in \mathbb{Z}$, and unrestricted real y give an infeasible integer problem, although minimizing y in the relaxation is unbounded. Production solvers must distinguish this from the finite-node-LP case above.

5. What a solver's bounds and gap actually mean

The bound of the **current node** is not necessarily a global lower bound. Other open branches may contain cheaper solutions. Let $\mathcal{A}$ be the open frontier, including any node being processed, and let ℓ_N be each node's certified bound. Until solved, a child can safely retain its parent's bound. Define

$$L = \min\left\{U, \min_{N \in \mathcal{A}} \ell_N\right\}, \quad L \le z^\star \le U. \tag{8}$$

Take the minimum over an empty frontier as $+\infty$. Then $L = U$ on exhaustion if an incumbent exists. Including U avoids reporting a "lower bound" above the already solved optimum when all remaining nodes are worse.

For our five-node traversal, if only the selected node's LP is solved:

Checkpoint	**Global L**	**Incumbent U**	**Absolute gap**
Root solved	$\frac{25}{3}$	10	$\frac{5}{3}$
B, C, D finished; A still open	$\frac{25}{3}$	10	$\frac{5}{3}$
A solved; frontier empty	9	9	0

The middle row is intentionally unchanged: A has not been solved, so it still carries the root bound. Knowing that B costs at least 9 does not prove that A does. Solving both child relaxations earlier would permit $L = 9$ sooner. With exact nested relaxations, inherited bounds never decrease, and U never increases; this makes the **absolute** gap nonincreasing.

5.1. Stop early with a meaningful guarantee

For finite bounds, if $U - L \le \varepsilon$, then

$$0 \le U - z^\star \le U - L \le \varepsilon. \tag{9}$$

This is an absolute suboptimality guarantee in the objective's units. A common relative gap is $\frac{|U-L|}{|U|}$ for $U \neq 0$. Solvers have their own conventions at zero and infinity; check the documentation. Relative gaps can even increase as absolute gaps decrease when the incumbent approaches zero. Adding a constant to the objective changes the relative gap but not the optimizer or absolute gap. The log's **Incumbent**, **BestBd**, and **Gap** are the practical interface to these certificates [2].

At a time limit, report the feasible incumbent, the best global bound, the remaining gap, and the termination status. With no incumbent, no implementable solution has been certified. "Not proven optimal" does not mean "infeasible."

5.2. A useful strengthening when the objective is integer-valued

If **every feasible objective value is an integer**, a valid bound L improves to $\lceil L \rceil$. Our root bound becomes $\left\lceil \frac{25}{3} \right\rceil = 9$. Thus **any** feasible point of cost 9 already proves optimality, even with open nodes. Do not apply this when continuous variables or nonintegral costs destroy objective integrality.

5.3. Floating point is not exact arithmetic

Implementations use feasibility, integrality, and gap tolerances. A value near an integer is not automatically a feasible rounded solution; substitute the candidate into the original constraints. Poor scaling and very large big-M constants can make tolerance effects consequential. Solver-certified optimality is normally relative to these numerical tolerances; exact rational certificates or independently checked bounds are needed for an exact proof.

6. How to make the search smaller

Branch and bound separates **finding an answer** from **proving that nothing is better**. Better incumbents push U down. Stronger relaxations push L up. Either can close the same gap, but only the second addresses a weak proof bound.

6.1. Stronger formulations and cutting planes

A **valid inequality** keeps all original-feasible points but may remove fractional ones. Adding the two service constraints in Equation 2 gives

$$3x + 3y \geq 10, \quad x + y \geq \frac{10}{3}. \tag{10}$$

Because $x + y$ is an integer at every original-feasible point,

$$x + y \geq 4 \tag{11}$$

is valid. It removes the root fractional solution, whose coordinate sum is $\frac{10}{3} < 4$. This is a simple rounded aggregation (a Chvátal–Gomory cut). With Equation 11, the root LP already solves the integer problem:

$$3x + 2y = (2x + y) + (x + y) \geq 5 + 4 = 9. \tag{12}$$

The point $(1, 3)$ attains 9. One strengthened LP replaces the five-node tree. **Branch-and-cut** adds valid inequalities during branch and bound; this is the basic architecture of modern MILP solvers [1].

> A cut removes fractional artifacts without removing legal decisions. A branch splits legal decisions into separate subproblems. Neither operation is allowed to delete a potentially better feasible answer.

Avoid unnecessarily large big-M values in logical models. For example, the activation relation $0 \leq w \leq Mb$, $b \in \{0, 1\}$, relaxes to $w \leq Mb$ with $0 \leq b \leq 1$. If useful output $w = 10$ is possible and $M = 10^6$, the LP can "activate" it with $b = 10^{-5}$, severely underpaying any activation cost. Tight valid bounds or suitable indicator formulations can be much stronger.

6.2. Two different selection decisions

- **Which open node next?** Depth-first search uses little frontier memory and often reaches candidate solutions quickly. Best-bound search chooses the smallest ℓ_N to focus on the global proof bottleneck; it may store many nodes.
- **Which variable to branch on?** A simple rule chooses a fractional component close to a half-integer. Strong branching tentatively solves candidate child LPs; pseudocosts estimate their bound gains from earlier branches. Stronger decisions cost more per node, so fewer nodes need not mean less time.

Warm starts, presolve, domain propagation, primal heuristics, and cuts all help. Node count alone is not a solver-independent performance measure.

6.3. Beyond MILP: the bound must remain global

For convex mixed-integer models, convex relaxations or valid dual bounds can replace LPs. In nonconvex nonlinear problems, merely dropping integrality and running a local optimizer does **not** yield a valid lower bound: a local minimum may be above a better feasible point. Spatial branch and bound subdivides continuous domains and uses globally valid underestimators; convergence also needs bounds that tighten as regions shrink [3].

In robotics, a discrete obstacle-side, contact-mode, or task-assignment choice can sit above a continuous trajectory problem. Branching organizes these choices; convex subproblems may bound them. A locally solved nonlinear trajectory is a candidate, not a global bound. In real-time mixed-integer MPC, a small cost gap does not by itself certify closed-loop safety or recursive feasibility.

7. Reproduce the numbers, then test your understanding

The accompanying `scripts/verify.m` uses Octave's existing GLPK interface; no additional package is required. From the repository root:

```
octave --no-gui --quiet Optimization/branch-and-bound/scripts/verify.m
typst compile Optimization/branch-and-bound/branch-and-bound.typ
```

It solves the five node relaxations and the cut-strengthened root; independently enumerates all 18 integer assignments in the box; compares each finite node bound with that node's true integer optimum; and checks the cut on every feasible integer assignment. Its expected headline results are:

Problem	**Checked result**
Root relaxation	$\left(\frac{5}{3}, \frac{5}{3}\right)$, value $\frac{25}{3}$
B: $x = 2$	$\left(2, \frac{3}{2}\right)$, value 9
C: $x = 2, y \leq 1$	Infeasible
D: $x = 2, y \geq 2$	$(2, 2)$, value 10
A: $x \leq 1$	$(1, 3)$, value 9
Root plus $x + y \geq 4$	$(1, 3)$, value 9
Full integer enumeration	Unique optimum $(1, 3)$, value 9

These are numerical cross-checks, not a replacement for the exact certificates derived above. The script checks the educational example; it is not a production branch-and-bound implementation.

7.1. Questions that expose common mistakes

1. **Min versus max.** For a maximization problem, which bound comes from a relaxation, and when can a node be pruned?
2. **A misleading current node.** The incumbent is 100. Two open minimization nodes have bounds 95 and 70. Can you claim an absolute gap of 5?
3. **Equality.** If $\ell_N = U$, can you close the node when seeking one optimum? What changes when enumerating all optima?
4. **A failed rounding heuristic.** If every rounded LP point is infeasible, what can you conclude about the original problem?
5. **A new objective.** Replace the cost by $3x + 2.1y$. Can you still strengthen the root bound simply by taking its ceiling?

Answers. (1) The relaxation supplies an upper bound; the incumbent supplies a lower bound. Prune when the node's upper bound is no greater than the incumbent. (2) No: the global bound is 70, so the certified absolute gap is 30. (3) Yes for one optimum; not in general for all optima, since ties can live there. (4) Nothing about infeasibility: only that the heuristic failed. (5) No: integer decisions no longer guarantee integer-valued costs; for example, $(1, 3)$ costs 9.3. A different known objective lattice could support a different rounding rule, but not the integer ceiling rule as stated.

> **The method in one sentence:** maintain a feasible incumbent and valid bounds, partition every unresolved decision, and stop only when the remaining regions are proved irrelevant or the certified gap is small enough.

Sources and further reading

[1] A. Gleixner, "Computational Mixed-Integer Programming." [Online]. Available: https://www.birs.ca/cmo-workshops/2018/18w5208/files/GleixnerAmbros.pdf

[2] Gurobi Optimization, "MIP Logging." Accessed: Oct. 20, 2026. [Online]. Available: https://docs.gurobi.com/projects/optimizer/en/current/concepts/logging/mip.html

[3] S. Boyd and J. Mattingley, "Branch and Bound Methods." [Online]. Available: https://web.stanford.edu/class/ee364b/lectures/bb_notes.pdf